Benchmarking Filtering Techniques for Entity Resolution

George Papadakis
National and Kapodistrian University of Athens, Greece
gpapadis@di.uoa.gr

Marco Fisichella
L3S Research Center, Germany
mfisichella@l3s.de

Franziska Schoger
L3S Research Center, Germany
schoger@l3s.de

George Mandilaras
National and Kapodistrian University of Athens, Greece
gmandi@di.uoa.gr

Nikolaus Augsten
University of Salzburg, Austria
nikolaus.augsten@plus.ac.at

Wolfgang Nejdl
L3S Research Center, Germany
nejdl@l3s.de

2023 IEEE 39th International Conference on Data Engineering (ICDE) | 979-8-3503-2227-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICDE55515.2023.00389

***Abstract*—Entity Resolution is the task of identifying pairs of entity profiles that represent the same real-world object. To avoid checking a quadratic number of entity pairs, various filtering techniques have been proposed that fall into two main categories: (i) *blocking workflows* group together entity profiles with identical or similar signatures, and (ii) *nearest-neighbor methods* convert all entity profiles into vectors and identify the closest ones to every query entity. Unfortunately, the main techniques from these two categories have rarely been compared in the literature and, thus, their relative performance is unknown. We perform the first systematic experimental study that investigates the relative performance of the main representatives per category over numerous established datasets. Comparing techniques from different categories turns out to be a non-trivial task due to the various configuration parameters that are hard to fine-tune, but have a significant impact on performance. We consider a plethora of parameter configurations, optimizing each technique with respect to recall and precision targets. Both schema-agnostic and schema-based settings are evaluated. The experimental results provide novel insights into the effectiveness, the time efficiency and the scalability of the considered techniques.**

***Index Terms*—nearest neighbors, record linkage, deduplication**

I. Introduction

Entity Resolution (ER) is a well-studied problem that aims to identify so-called *duplicates* or *matches*, i.e., different entity profiles that describe the same real-world object [1]. ER constitutes a crucial task in a number of data integration tasks, which range from Link Discovery for interlinking the sources of the Linked Open Data Cloud to data analytics, query answering and object-oriented searching [2]–[4].

Due to its quadratic time complexity, ER typically scales to large data through a Filtering-Verification framework [1], [3]. The first step (*filtering*) constitutes a coarse-grained, rapid phase that restricts the computational cost to the most promising matches, a.k.a. *candidate pairs*. This is followed by the *verification* step, which examines every candidate pair to decide whether it is a duplicate.

Verification is called *matching* in the ER literature [2], [3], [5]. Numerous matching methods have been proposed; most of them rely on similarity functions that compare the textual values of entity profiles [2], [3], [5], [6]. Early attempts were rule-based, comparing similarity values with thresholds, but more recent techniques rely on learning, i.e., they usually model matching as a binary classification task (`match`, `non-match`) [6]. Using a labelled training dataset, supervised, active, and deep learning techniques are adapted to ER [7]. In some cases, a clustering step is subsequently applied on the resulting similarity scores to refine the output [8].

In this work, we are interested in filtering methods, which significantly reduce the search space of ER. The performance of a filtering technique is assessed along three dimensions: recall, precision and time efficiency. A good filter stands out by:

1) *High recall.* The candidate pairs should involve many duplicates to reduce the number of false negatives.
2) *High precision.* The candidate pairs should involve few false positives (i.e., non-matching pairs) to significantly reduce the search space.
3) *Low run-time.* The overhead added by the filtering step to the overall run-time of ER should be low.

Ideally, filtering should also be directly applicable to the input data. For this reason, we exclusively consider methods that require no labelled training instances, which are often not available or expensive to produce [9]. We also focus on techniques for textual entity profiles. These methods are organized into two types [10]:

1) *Blocking workflows* extract one or more signatures from every entity profile and form clusters of profiles with identical or similar signatures [3], [11], [12].
2) *Nearest neighbor (NN) methods* index part or all of the input data and treat every entity profile as a query, to which they return the most similar candidates. They rely on sparse [13], [14] or dense numeric vectors [15].

Although the two types of filtering techniques follow very different approaches, they all receive the same input (the entity profiles) and produce the same output (candidate pairs).

To the best of our knowledge, no previous work systematically examines the relative performance across the two different kinds of methods. The main blocking methods are empirically evaluated in [11], [12], [16]. The studies on sparse [13], [14], [17] and dense vector-based NN methods [15] evaluate run-time and approximation quality, but do not evaluate the performance on ER tasks (cf. Section II). Even in the few cases where blocking is used as baseline for an NN method (e.g., Standard/Token Blocking is compared to DeepBlocker in [9]), the comparison is not fair: the blocking method is treated

2375-026X/23/$31.00 ©2023 IEEE
DOI 10.1109/ICDE55515.2023.00389

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

as an independent approach, instead of applying it as part of a blocking workflow, which is common in the literature [11].

Comparing techniques from different categories turns out to be a challenging task, due to their fundamentally different functionality and the diversity of configuration parameters that significantly affect their performance. Yet, there is *no* systematic fine-tuning approach that is generic enough to apply to all filtering methods – e.g., the step-by-step configuration optimization in [11] applies to blocking pipelines, but not to the single-stage NN methods.

In this work, we perform the first thorough and systematic experimental study that covers both types of filtering methods. To ensure a fair comparison, every approach is represented by its best performance per dataset, as it is determined after an exhaustive fine-tuning with respect to a common performance target that considers several thousand different parameter configurations. This approach is applied to 5 blocking workflows and 8 NN methods over 10 real-world datasets. For each filter type, we also consider two baseline methods with default parameters. We explore both *schema-agnostic* and *schema-based settings*: the latter focus exclusively on the values of the most informative attribute, while the former take into account all information within an entity, essentially treating it as a long textual value – as a result, they are inherently applicable to datasets with heterogeneous schemata.

Overall, we make the following contributions:

- We perform the first thorough experimental analysis on filtering techniques from different categories. We evaluate 14 state-of-the-art filters and 4 baselines on 10 real-world datasets in both schema-based and schema-agnostic settings.
- Our configuration optimization process enables the meaningful comparison of blocking workflows and NN methods in the context of serial processing (single-core execution).
- We present a qualitative analysis of the filtering techniques based on their *scope* and *internal functionality*.
- We perform a thorough scalability analysis that involves seven synthetic datasets of increasing size.
- Our work provides new insights into the relative performance and scalability of the considered techniques. We show that the blocking workflows and the cardinality-based sparse NN methods consistently excel in performance.
- Two of the tested NN methods, SCANN and kNN-Join, are applied to ER for the first time. SCANN is one of the most scalable techniques, and kNN-Join one of the best performing ones, while sticking out by its intuitive fine-tuning.
- All code and data used in this work are publicly available through a new, open initiative that is called Continuous Benchmark of Filtering methods for ER: https://github.com/gpapadis/ContinuousFilteringBenchmark.

The main part of the paper is structured as follows: Section II discusses the related works, while Section III defines formally the configuration optimization task. We elaborate on the filtering methods in Section IV and present their qualitative and quantitative analyses in Sections V and VI, resp. Section VII concludes with the main findings of our experimental analysis along with directions for future research.

II. Related Work

There has been a plethora of works examining the relative performance of blocking methods. The earliest systematic studies were presented in [3], [12]. They focus exclusively on schema-based settings in combination with several user-defined parameter configurations. They also consider exclusively the first step of blocking workflows: block building.

These studies were extended in [16], which examines the same block building methods and configurations, but applies them to schema-agnostic settings, too. The experimental outcomes suggest that recall raises significantly, when compared to the schema-based settings, while requiring no background knowledge about the given data and the quality of its schema. They also suggest that the sensitivity to parameter configuration is significantly reduced.

Building on these works, the experimental analysis in [11] examines the relative performance of the blocking workflows in schema-agnostic settings. In particular, it considers blocking workflows formed by exactly three steps: block building, block filtering and comparison cleaning. In our work, we extend this analysis by considering the top performing block building methods, all of which cluster together entities that share identical signatures. We combine them with three consecutive, but optional steps: block purging, block filtering and comparison cleaning. These steps give rise to seven different filtering pipelines, out of which only one was examined in [11].

Our work also differs from [11] in that it has only one dataset in common – excluding the scalability ones. Most importantly, [11] optimizes the configuration parameters in a heuristic step-by-step manner: first, the performance of block building is heuristically optimized and then, block filtering is heuristically fine-tuned by receiving as input the output of optimized block building as so on for comparison cleaning. In contrast, we consider a holistic approach to configuration optimization, simultaneously fine-tuning all steps in a blocking workflow. As explained in [18], [19], this approach consistently outperforms the step-by-step fine-tuning, because it is not confined to local maxima per workflow step, while it considers a significantly larger set of possible configurations.

Finally, we also go beyond [3], [11], [11], [12] in two more ways: (i) we systematically fine-tune blocking workflows in the context schema-based settings, and (ii) we compare blocking workflows with sparse and dense NN methods.

The sparse vector-based NN methods essentially correspond to similarity joins. The relative performance of the main methods is examined in [13], [14], [17] with respect to runtime. Recall and precision are not considered, because they are identical across all approaches, i.e., all methods retrieve the same pairs of entities that exceed a similarity threshold. These pairs, which are not necessarily matching, have not been evaluated with respect to the recall and precision of ER. As a result, these studies are not useful in assessing the performance of string similarity joins for ER. Note also that none of the sparse NN methods we consider has been examined in prior experimental analyses. The reason is that the local kNN-Join

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

lies out of the focus of [13], [14], [17], while the range joins for ER involve very low similarity thresholds, unlike the string similarity joins examined in [13], [14], [17].

The dense vector-based NN methods are experimentally compared in [15], but the evaluation measures are restricted to throughput, i.e., executed queries per second, and to recall, i.e., the portion of retrieved vectors that are indeed the nearest ones with respect to a specific distance function (e.g., the Euclidean one). This is different from ER recall, as the closest vectors are not necessarily matching. Nevertheless, we rely on the experimental results of [15] in order to select the top-performing dense NN methods: Cross-polytope and Hyperplane LSH, FAISS and SCANN. We also consider MinHash LSH, a popular filtering technique [10], and DeepBlocker, the most recent learning-based approach, which consistently outperforms all others [9].

III. Preliminaries

We define an entity profile e_i as the set of textual name-value pairs, i.e., $e_i = \{\langle n_j, v_j \rangle\}$, that describes a real-world object [11], [16]. This model covers most established data formats, such as the structured records in relational databases and the semi-structured instance descriptions in RDF data. Two entities, e_i and e_j, that pertain to the same real-world object ($e_i \equiv e_j$) are called *duplicates* or *matches*.

ER is distinguished into two main tasks [3], [5]:

1) *Clean-Clean ER* or *Record Linkage*, which receives as input two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$ that are individually duplicate-free, but overlapping, and
2) *Dirty ER* or *Deduplication*, whose input comprises a single set of entity profiles, $\mathcal{E}$, with duplicates in itself.

In both cases, the output consists of the detected duplicate profiles. In the context of Clean-Clean ER, the filtering methods receive as input $\mathcal{E}_1$ and $\mathcal{E}_2$ and produce a set of candidate pairs $\mathcal{C}$, which are highly likely to be duplicates and should be analytically examined during the verification step. To measure the **effectiveness** of filtering, the following measures are typically used [5], [11], [12], [16], [20]:

1) *Pair Completeness (PC)* expresses *recall*, estimating the portion of the duplicate pairs in $\mathcal{C}$ with respect to those in the groundtruth: $PC(\mathcal{C}, \mathcal{E}_1, \mathcal{E}_2) = |\mathcal{D}(\mathcal{C})|/|\mathcal{D}(\mathcal{E}_1 \times \mathcal{E}_2)|$, where $\mathcal{D}(x)$ denotes the set of duplicates in set x.
2) *Pairs Quality (PQ)* captures *precision*, estimating the portion of comparisons in $\mathcal{C}$ that correspond to real duplicates: $PQ(\mathcal{C}) = |\mathcal{D}(\mathcal{C})|/|\mathcal{C}|$.

All measures result in values in the range $[0, 1]$, with higher values indicating higher effectiveness. Note that there is a trade-off between PC and PQ: the larger C is, the higher PC gets at the cost of lower PQ, and vice versa for a smaller set of candidates. The goal of filtering is to achieve a good balance between these measures.

In this context, we formalize the following configuration optimization task, which enables the comparison of fundamentally different filtering techniques on an equal basis:

Problem 1 (Configuration Optimization): Given two sets of entity profiles, $\mathcal{E}_1$ and $\mathcal{E}_2$, a filter method, and a threshold τ on recall (PC), configuration optimization fine-tunes the parameters of the filtering method such that the resulting set of candidates $\mathcal{C}$ on $\mathcal{E}_1$ and $\mathcal{E}_2$ maximizes PQ for $PC \geq \tau$.

Note that we set a threshold on recall, because ER solutions typically consist of two consecutive steps: Filtering and Matching. The recall of the former step determines the overall recall of ER, since the duplicate pairs that are not included in the resulting set of candidates C cannot be detected by most matching methods. This applies both to Clean-Clean ER, where there is no transitivity, due to the 1-1 matching constraint, and to Dirty ER, where the matching algorithms typically consider local information, e.g., the (deep) learning-based methods that treat matching as a binary classification task [21], [22]. As a result, we chose $\tau = \mathbf{0.9}$ as the PC threshold that ensures high recall for the overall ER process. Preliminary experiments demonstrated that a lower threshold, e.g., 0.85, didn't alter the relative performance of the considered techniques. Besides, in more than 40% of considered cases, the filtering techniques address Problem 1 with $PC \geq 91\%$, i.e., much higher than our threshold.

Regarding **time efficiency**, the *run-time* (RT) measures the time between receiving the set(s) of entity profiles as input and producing the set of candidate pairs as output. RT should be minimized to restrict the overhead of filtering on ER.

IV. Filtering Methods

A. The two paradigms of filtering

Blocking methods first associate every input entity with one or (usually) more signatures and then, they cluster together entities with identical or similar signatures into blocks. Every pair of entities that appears in at least one block is considered a matching candidate. The resulting blocks contain two unnecessary types of candidates, i.e., pairs whose verification lowers precision, without any benefit for recall:

1) The *redundant* candidates are repeated across different blocks, because every entity typically participates in multiple ones and, thus, the blocks are overlapping.
2) The *superfluous* candidates involve non-matching entities.

To eliminate the former and reduce the latter, block and comparison cleaning are applied to the initial blocks, restructuring them based on global patterns [11], [23]. Figure 1 depicts the complete workflow for blocking [10], [24]. Initially, a set of blocks is created by at least one block building method. The initial block collection is then processed by two coarse-grained block cleaning techniques, Block Purging and Block Filtering. Both produce a new, smaller block collection, B' and B'', respectively, but are optional and can be omitted, e.g., in the case of schema-based blocks with low levels of redundant and/or superfluous comparisons. Finally, a comparison cleaning technique is applied, whose fine-grained functionality decides for individual comparisons whether they should be retained or discarded; in this mandatory step, at least the redundant candidates are discarded, but the superfluous ones are also subject to removal.

A fundamentally different approach is followed by NN methods. Instead of extracting signatures from the input entity

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

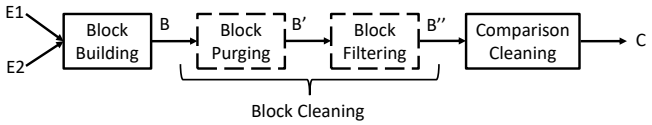


Fig. 1. The blocking workflow [24]. Dotted contours indicate optional steps.

profiles, they organize the input set $\mathcal{E}_1$ into an index I (e.g., an inverted index) and use the other dataset as a query set, as shown in Figure 2. This means that the set of candidate pairs C is formed by probing the index I for every entity profile $e_2 \in \mathcal{E}_2$ and aggregating all query results. To restrict the noise in this process, the textual attribute values are typically cleaned from stop-words and every word is reduced to its base/root form through lemmatization or stemming (e.g., "blocks" becomes "block") [25]. The attribute values may also be transformed into (pre-trained) embeddings, i.e., into fixed-length, dense and distributed representations that give rise to semantic similarities [26]. These optional, preprocessing steps apply to both inputs. After cleaning, we get $\mathcal{E}_1'$ and $\mathcal{E}_2'$, which remain in textual form, but after embedding, we get two sets of dense numeric vectors, $\mathcal{VE}_1$ and $\mathcal{VE}_2$, respectively.

Note that the blocking workflows produce redundant candidates as intermediate results of block building, which are eliminated during comparison cleaning. Contrariwise, NN methods produce no redundant candidates, as every query entity from $\mathcal{E}_2$ is associated with a subset of the indexed entities from $\mathcal{E}_1$.

B. Blocking workflows

Block building. We consider the following state-of-the-art techniques, based on the results of the past experimental studies [11], [12], [16],

1) *Standard Blocking.* Given an entity, it tokenizes the considered attribute values on whitespace and uses the resulting tokens as signatures. Hence, every block corresponds to a distinct token, involving all entities that contain it in the selected attribute value(s).

2) *Q-Grams Blocking.* To accommodate character-level errors, it defines as signatures the set of q-grams that are extracted from the tokens of Standard Blocking. In other words, every block corresponds to a distinct q-gram, encompassing all entities with that q-gram in any of the considered values.

3) *Extended Q-Grams Blocking.* Instead of individual q-grams, the signatures of this approach are constructed by concatenating at least L q-grams, where $L = max(1, \lfloor k \cdot t \rfloor)$, k is the number of q-grams extracted from the original key/token and $t \in [0, 1)$ is a threshold that reduces the number of combinations as its value increases. Compared to Q-Grams Blocking, the resulting blocks are smaller, but contain candidate pairs that share more content.

4) *Suffix Arrays Blocking.* Another way of accommodating character-level errors in the signatures of Standard Blocking is to consider their suffixes, as long as they comprise a minimum number of characters l_{min}. Every block corresponds to a token suffix that is longer than l_{min} and appears in $< b_{max}$ entities.

5) *Extended Suffix Arrays Blocking.* This approach generalizes the previous one by converting the signatures of Standard Blocking in all substrings longer than l_{min} and less frequent than b_{max} entities.

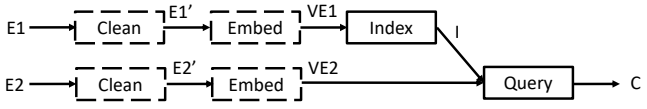


Fig. 2. The workflow of NN methods. Dotted contours indicate optional steps.

Example. To illustrate the difference between these blocking methods, consider as an example the attribute value "Joe Biden". Standard Blocking produces 2 blocking keys: {Joe, Biden}. With $q = 3$, Q-Grams Blocking produces 4 keys: {Joe, Bid, ide, den}. For T=0.9, Extended Q-Grams Blocking combines at least two q-grams from each token, defining the following 5 blocking keys: {Joe, Bid_ide_den, Bid_ide, Bid_den, ide_den}. Using l_{min}=3 and a large enough b_{max}, Suffix Arrays Blocking yields 4 keys: {Joe, Biden, iden, den}, while Extended Suffix Arrays extracts 7 keys: {Joe, Biden, Bide, iden, Bid, ide, den}.

Note that we exclude Attribute Clustering Blocking [27], because it is incompatible with the schema-based settings we are considering in this work. Note also that *we experimented with Sorted Neighborhood [3], [12], [16], but do not report its performance, since it consistently underperforms the above methods*. The reason is that this method is incompatible with block and comparison cleaning techniques that could reduce its superfluous comparisons [10], [11].

Block cleaning. We consider two complementary methods:

1) *Block Purging [27].* This parameter-free approach assumes that the larger a block is, the less likely it is to convey matching pairs that share no other block. Such blocks emanate from signatures that are stop-words. Therefore, it removes the largest blocks (e.g., those containing more than half the input entities) in order to significantly increase precision at a negligible (if any) cost in recall.

2) *Block Filtering [16].* It assumes that for a particular entity e, its largest blocks are less likely to associate e with its matching entity. For every entity e, it orders its blocks in increasing size and retains it in $r\%$ of the top (smaller) ones – r is called filtering ratio. This increases precision to a significant extent for slightly lower recall.

Comparison cleaning. We consider two established methods, but only one can be applied in a blocking workflow [11], [24]:

1) *Comparison Propagation [27].* This parameter-free approach removes all redundant pairs from any block collection without missing any matches, i.e., it increases precision at no cost in recall. It associates every entity with the list of its block ids and retains every candidate pair only in the block with the least common id.

2) *Meta-blocking [28].* It targets both redundant and superfluous comparisons using (i) a *weighting scheme*, which associates every candidate pair with a numerical score proportional to the matching likelihood that is extracted from the blocks shared by its constituent entities, and (ii) a *pruning algorithm*, which leverages these scores to decide which candidate pairs will be retained in the restructured block collection that is returned as output.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

The rationale behind weighting schemes is that the more and smaller blocks two entities share (i.e., the more and less frequent signatures they share), the more likely they are to be matching. In this context, the following schemes have been proposed [10], [28], [29]: ARCS promotes pairs that share smaller blocks; CBS counts the blocks the two entities have in common; ECBS extends CBS by discounting the contribution of entities participating in many blocks; JS computes the Jaccard coefficient of the block ids associated with the two entities; EJS extends JS by discounting the contribution of entities participating in many non-redundant pairs; χ^2 estimates to which degree the two entities appear independently in blocks.

These schemes can be combined with the following pruning algorithms [28], [29]: BLAST retains a pair if its weight exceeds the average maximum weight of its constituent entities; CEP and CNP retain the overall top-K pairs and the top-k pairs per entity, respectively (K and k are automatically configured according to input blocks characteristics); Reciprocal CNP (RCNP) requires that every retained pair is ranked in the top-k positions of both constituent entities; WEP discards all pairs with a weight lower than the overall average one; WNP keeps only pairs with a weight higher than the average one of at least one of their entities; Reciprocal WNP (RWNP) requires a weight higher than the average of both entities.

C. Sparse vector-based NN methods

This type includes set-based similarity joins methods, which represent each entity by a set of *tokens* such that the similarity of two entities is derived from their token sets. The similarity between two token sets A and B is computed through one of the following measures, normalized in $[0, 1]$ [13]:

1) Cosine similarity $C(A, B) = |A \cap B|/\sqrt{|A| \cdot |B|}$.
2) Dice similarity $D(A, B) = 2 \cdot |A \cap B|/(|A| + |B|)$.
3) Jaccard coefficient $J(A, B) = |A \cap B|/|A \cup B|$.

The tokens are extracted from string attributes (the concatenation of all attribute values in the schema-agnostic settings) by considering the character n-grams [30] (as in Q-Grams Blocking) or by splitting the strings on whitespace (as in Standard Blocking). Duplicate tokens within one string are either ignored or de-duplicated by attaching a counter to each token [31] (e.g., $\{a, a, b\} \rightarrow \{a_1, a_2, b_1\}$).

Candidate pairs are formed based on the similarity of two entities according to some *matching principles* [32]. We combine two well-known principles with all the aforementioned similarity measures and tokenization schemes [33]:

1) *Range join (ε-Join) [33].* It pairs all entities that have a similarity no smaller than a user-defined threshold ε. Numerous efficient algorithms for ε-Join between two collections of token sets have been proposed [13], [34]–[40]. All of these techniques produce the exact same set of candidates, but most of them are crafted for high similarity thresholds (above 0.5), which is not the case in ER, as shown in Table X in [41]. For this reason, we employ *ScanCount* [42], which is suitable for low similarity thresholds. In essence, it builds an inverted list on all tokens in the entity collection $\mathcal{E}_1$ and for the lookup of a query entity/token set $e_i \in \mathcal{E}_2$, it performs merge-counts on the posting lists of all tokens in e_i. Then, it returns all pairs that exceed the similarity threshold ε.

2) *k-nearest-neighbor join (kNN-Join) [33].* Given two collections, $\mathcal{E}_1$ and $\mathcal{E}_2$, it pairs each entity in $e_i \in \mathcal{E}_2$ with the k most similar elements in $\mathcal{E}_1$ that have distinct similarity values, i.e., e_i may be paired with more than k entities if some of them are equidistant from e_i. The kNN-Join is not commutative, i.e., the order of the join partners matters. An efficient technique that leverages an inverted list on tokens that are partitioned into size stripes is the *Cone* algorithm [43], which is crafted for label sets in the context of top-k subtree similarity queries. To increase the limited scope of the original algorithm, we adapted it to leverage ScanCount.

Note that the top-k set similarity joins [44], [45] compute the k entity pairs between $\mathcal{E}_1$ and $\mathcal{E}_2$ with the highest similarities among all possible pairs. This means that they perform a global join that returns the k top-weighted pairs. This is equivalent to ε-Join, if the kth has a similarity equal to ε. Instead, the kNN-Join performs a local join that returns at least k pairs per element $e_i \in \mathcal{E}_2$.

D. Dense vector-based NN methods

LSH. Locality Sensitive Hashing [46], [47] constitutes an established solution to the approximate nearest neighbor problem in high-dimensional spaces. Its goal is to find entities/vectors that are within $c \cdot R$ distance from a query vector, where $c > 1$ is a real number that represents a user-specified approximation ratio, while R is the maximum distance of any nearest neighbor vector from the query. LSH is commonly used as a filtering technique for ER [48]–[51] because of its sub-linear query performance, which is coupled with a fast and small index maintenance, and its mathematical guarantee on the query accuracy. We consider three popular versions:

1) *MinHash LSH (MH-LSH)* [52], [53]. Given two token sets, this approach approximates their Jaccard coefficient by representing each set as a minhash, i.e., a sequence of hash values that are derived from the minimum values of random permutations. The minhashes are decomposed into a series of bands consisting of an equal number of rows. This decomposition has a direct impact on performance: if there are few bands with many rows, there will be collisions between pairs of objects with a very high Jaccard similarity; in contrast, when there are many bands with few rows, collisions occur between pairs of objects with very low similarity. The selected number of bands ($\#bands$) and rows ($\#rows$) approximates a step function, i.e., a high-pass filter, which indicates the probability that two objects share the same hash value: $(1/\#bands)^{(1/\#rows)}$.

2) *Hyperplane LSH (HP-LSH)* [54]. The vectors are assumed to lie on a unit hypersphere divided by a random hyperplane at the center, formed by a randomly sampled normal vector r. This creates two equal parts of the hypersphere with $+1$ on the one side and -1 on the other. A vector v is hashed into $h(v) = sgn(r \cdot v)$. For two vectors v_1 and v_2 with an angle α between them, the probability of collision is $Pr[h(v_1) = h(v_2)] = 1 - \frac{\alpha}{\pi}$

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE SCOPE PER TYPE OF FILTERING METHODS.

Scope		Blocking	Sparse NN	Dense NN
Syntactic Representation	Schema-based	✓	✓	✓
	Schema-agnostic	✓	✓	✓
Semantic Representation	Schema-based	-	-	✓
	Schema-agnostic	-	-	✓

3) *Cross-Polytope LSH (CP-LSH)* [55]. It is a generalization of HP-LSH. At their core, both HP- and CP-LSH are random spatial partitions of a d-dimensional unit sphere centered at the origin. The two hash families differ in how granular these partitions are. The cross-polytope is also known as an l1-unit ball, where all vectors on the surface of the cross-polytope have the l1-norm. In CP-LSH, the hash value is the vertex of the cross-polytope closest to the (randomly) rotated vector. Thus, a cross-polytope hash function partitions the unit sphere according to the Voronoi cells of the vertices of a randomly rotated cross-polytope. In the 1-dimensional case, the cross-polytope hash becomes the hyperplane LSH family.

kNN-Search. We consider three popular frameworks:

1) *FAISS [56].* This framework provides methods for kNN searches. Given two sets of (embedding) vectors, it associates every entry q from the query set with the k entries from the indexed set that have the smallest distance to q. Two approximate methods are provided: (i) a hierarchical, navigable small world graph method, and (ii) a cell probing method with Voroni cells, possibly in combination with product quantization. We experimented with both of them, but they do not outperform the Flat index with respect to Problem 1. The Flat index is also recommended by [56]. For these reasons, we exclude the approximate methods in the following. Note that FAISS also supports range, i.e., similarity, search, but our experiments showed that it consistently underperforms kNN search.

2) *SCANN [57].* This is another versatile framework with very high throughput. Two are the main similarity measures it supports: dot product and Euclidean distance. It also supports two types of scoring: brute-force, which performs exact computations, and asymmetric hashing, which performs approximate computations, trading higher efficiency for slightly lower accuracy. In all cases, SCANN leverages partitioning, splitting the indexed dataset into disjoint sets during training so that every query is answered by applying scoring to the most relevant partitions.

3) *DeepBlocker [9].* It is the most recent method based on deep learning, consistently outperforming all others, e.g., AutoBlock [50] and DeepER [51]. It converts attribute values into embedding vectors using fastText and performs indexing and querying with FAISS. Its novelty lies in the tuple embedding module, which converts the set of embeddings associated with an individual entity into a representative vector. Several different modules are supported, with the Autoencoder constituting the most effective one under the schema-based settings. In the schema-agnostic settings, the Autoencoder ranks second, lying in close distance of the top-performing Hybrid module, which couples Autoencoder with cross-tuple training.

Note that FAISS and SCANN also use 300-dimensional fastText embeddings. In fact, they are equivalent to the simple average tuple embedding module of DeepBlocker.

TABLE II
FUNCTIONALITY PER NN METHOD.

Operation	Similarity Threshold	Cardinality Threshold
Deterministic	ε-Join	kNN-Join, FAISS, SCANN
Stochastic	MH-, HP-, CP-LSH	DeepBlocker

V. QUALITATIVE ANALYSIS

Taxonomies. To facilitate the use and understanding of filtering methods, we organize them into two novel taxonomies.

Scope. The first taxonomy pertains to scope, i.e., the entity representation that lies at the core of the filtering method:

1) The *syntactic or symbolic representations* consider the actual text in an entity profile, leveraging the co-occurrences of tokens or character n-grams.
2) The *semantic representations* consider the embedding vectors that encapsulate a textual value, leveraging character-, word- or transformer-based models. We exclusively consider the unsupervised, pre-trained embeddings of fastText [58] that have been experimentally verified to effectively address the out-of-vocabulary cases in ER tasks, due to domain-specific terminology [9], [59]–[62].

These types are combined with schema-based and schema-agnostic settings, yielding the four fields of scope in Table I.

The distinctions introduced by this taxonomy are crucial for two reasons: (i) Syntactic representations have the advantage of producing intelligible and interpretable models. That is, it is straightforward to justify a candidate pair, unlike the semantic representations, whose interpretation is obscure to non-experts. (ii) Semantic representations involve a considerable overhead for transforming the textual values into embeddings, even when using pre-trained models. They also require external resources, which are typically loaded in main memory, increasing space complexity. Instead, the methods using syntactic representations are directly applicable to the input data.

We observe that dense NN methods have the broadest scope, being compatible with all four combinations. The syntactic representations are covered by MinHash LSH; its dimensions stem from character k-grams, which are called *k-shingles* and are weighted according to term frequency [53]. All other dense NN methods employ semantic representations in the form of fixed-size numeric vectors that are derived from fastText.

The blocking and the sparse NN methods cover only the syntactic similarities, as they operate directly on the input data.

Internal functionality. The second taxonomy pertains to the internal functionality of filtering methods. Blocking techniques have been categorized into *lazy* and *proactive* (see [11] for more details). For NN methods, we define the taxonomy in Table II, which comprises two dimensions:

1) The *type of operation*, which can be deterministic, lacking any randomness, or stochastic, relying on randomness.
2) The *type of threshold*, which can be similarity- or cardinality-based. The former specifies the minimum sim-

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE CONFIGURATION SPACE PER BLOCKING WORKFLOW.

	Parameter	Domain
Common	Block Purging (BP)	{ -, ✓}
	Block Filtering ratio (BFr)	[0.025, 1.00] with a step of 0.025
	Weighting Scheme (WS)	{ARCS, CBS, ECBS, JS, EJS, χ^2}
	Pruning Algorithm (PA)	CP or {BLAST, CEP, CNP, RCNP, RWNP, WEP, WNP}
Standard Blocking	Block Building	parameter-free
	Maximum Configurations	3,440
Q-Grams Blocking	q	$[2, 6]$ with a step of 1
	Maximum Configurations	17,200
Extended Q-Grams Blocking	q	$[2, 6]$ with a step of 1
	t	$[0.8, 1.0)$ with a step of 0.05
	Maximum Configurations	68,800
(Ex.) Suffix Arrays Blocking	l_{min}	$[2, 6]$ with a step of 1
	b_{max}	$[2, 100]$ with a step of 1
	Maximum Configurations	21,285

ilarity of candidate pairs, while the latter determines the maximum number of candidates per query entity.

The distinctions introduced by this taxonomy are important for two reasons: (i) The stochastic methods yield slightly different results in each run, unlike the deterministic ones, which yield a stable performance. This is crucial in the context of Problem 1, which sets a specific limit on a particular evaluation measure. For this reason, we set the performance of stochastic methods as the average one after 10 repetitions. (ii) The configuration of cardinality-based methods is straightforward and can be performed a-priori, because it merely depends on the number of input entities. In contrast, the similarity-based methods depend on data characteristics – the distribution of similarities, in particular.

Configuration space. As explained in Section I, a major aspect of filtering techniques is the fine-tuning of their configuration parameters, which has a decisive impact on their performance. For this reason, we combine every method with a wide range of values for each parameter through grid search. The domains we considered per parameter and method are reported in Tables III, IV and V.

Starting with Table III, the common parameters of the lazy blocking workflows include the presence or absence of Block Purging and the ratio used by Block Filtering. For the latter, we examined at most 40 values in $[0, 1]$, with 1 indicating the absence of Block Filtering. Given that these two steps determine the upper bound of recall for the subsequent steps, we terminate their grid search as soon as the resulting PC drops below the target one (0.9) – in these cases, the number of tested configurations is lower than the maximum possible one. For comparison cleaning, all methods are coupled with the parameter-free Comparison Propagation (CP) or one of the 42 Meta-blocking configurations, which stem from the six weighting schemes and the seven pruning algorithms.

The Standard Blocking workflow involves only the common parameters, yielding the fewest configurations. The rest of the blocking workflows use the same settings as in [11]. Note that the proactive ones, which are based on Suffix Arrays Blocking, are not combined with any block cleaning method.

In Table IV, we notice that the common parameters of

TABLE IV
THE CONFIGURATION SPACE PER SPARSE NN METHOD.

	Parameter	Domain
Common	Cleaning (CL)	{ -, ✓}
	Similarity Measure (SM)	{Cosine, Dice, Jaccard}
	Representation Model (RM)	{T1G, T1GM, C2G, C2GM, C3G, C3GM, C4G, C4GM, C5G, C5GM}
ε-Join	Similarity threshold (t)	[0.00, 1.00] with a step of 0.01
	Maximum Configurations	6,000
kNN-Join	Candidates per query (K)	[1, 100] with a step of 1
	Reverse Datasets (RVS)	{ -, ✓}
	Maximum Configurations	12,000

TABLE V
THE CONFIGURATION SPACE PER DENSE NN METHOD.

	Parameter	Domain
Common	Cleaning (CL)	{ -, ✓}
MH-LSH	$\#bands$	$\#bands \in P(2), \#rows \in P(2)$:
	$\#rows$	$\#bands \times \#rows \in \{128, 256, 512\}$
	k	$[2, 5]$ with a step of 1
	Configurations	168
HP- & CP-LSH	$\#tables$	$2^n : n \in \{0, 9\}$
	$\#hashes$	$[1, 20]$ with a step of 1
	last cp dimension	$2^n : n \in \{0, 9\}$
	Configurations	400 (HP), 2,000 (CP)

(a) Threshold-based algorithms

	Parameter	Domain
Common	Rev. Datasets (RVS)	{ -, ✓}
	K	$[1, 5000]$ with an increasing step
FAISS	Max. Configurations	2,720
SCANN	index	{ AH, BF }
	similarity	{ DP, LP^2 }
	Max. Configurations	10,880
DeepBlocker	Max. Configurations	2,720

(b) Cardinality-based algorithms

set-based similarity joins include the absence or presence of cleaning (i.e., stop-word removal and stemming), the similarity measure as well as the representation model. For the last two parameters, we consider all options discussed in Section IV-C, i.e., three similarity measures in combination with 10 models: whitespace tokenization (T1G) or character n-grams (CnG), with $n \in \{2, 3, 4, 5\}$; for each model, we consider both the set and the multiset of its tokens, with the latter denoted by appending M at the end of its name (e.g., T1GM).

Additionally, ε-Join is combined with up to 100 similarity thresholds. We start with the largest one and terminate the grid search as soon PC drops below target recall. kNN-Join is coupled with at most 100 cardinality thresholds, starting from the smallest one and terminating the grid search as soon PC exceeds the target recall. Another crucial parameter for kNN-Join is RVS, which is true (✓) if $\mathcal{E}_2$ should be indexed and $\mathcal{E}_1$ should used as the query set, instead of the opposite. In theory, kNN-Join involves double as many configurations as ε-Join, but in practice its cardinality threshold does not exceed 26 (see Table X in [41]), thus reducing significantly the maximum number of its configurations.

The parameters of dense NN methods are listed in Table V. The common parameter is the absence or presence of cleaning. In MinHash LSH, the number of bands and rows are powers of two such that their product is also a power of two, i.e., 2^n with $n \in \{7, 8, 9\}$. For k-shingles, we considered four common values for k, i.e., $[2, 5]$. For Hyperplane and CrossPolytope LSH, we configure two parameters: (i) the number of hash tables

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
TECHNICAL CHARACTERISTICS OF THE REAL DATASETS FOR CLEAN-CLEAN ER IN INCREASING COMPUTATIONAL COST.

	$\mathbf{D_{c1}}$	$\mathbf{D_{c2}}$	$\mathbf{D_{c3}}$	$\mathbf{D_{c4}}$	$\mathbf{D_{c5}}$	$\mathbf{D_{c6}}$	$\mathbf{D_{c7}}$	$\mathbf{D_{c8}}$	$\mathbf{D_{c9}}$	$\mathbf{D_{c10}}$
E_1 / E_2	Rest. 1 / Rest. 2	Abt / Buy	Amazon / GB	DBLP / ACM	IMDb / TMDb	IMDb / TVDB	TMDb / TVDB	Walmart / Amazon	DBLP / GS	IMDb / DBpedia
E_1 / E_2 entities	339 / 2,256	1,076 / 1,076	1,354 / 3,039	2,616 / 2,294	5,118 / 6,056	5,118 / 7,810	6,056 / 7,810	2,554 / 22,074	2,516 / 61,353	27,615 / 23,182
Duplicates	89	1,076	1,104	2,224	1,968	1,072	1,095	853	2,308	22,863
Cartesian Product	$7.65{\cdot}10^5$	$1.16{\cdot}10^6$	$4.11{\cdot}10^6$	$6.00{\cdot}10^6$	$3.10{\cdot}10^7$	$4.00{\cdot}10^7$	$4.73{\cdot}10^7$	$5.64{\cdot}10^7$	$1.54{\cdot}10^8$	$6.40{\cdot}10^8$
Best Attribute	Name	Name	Title	Title	Title	Name	Name	Title	Title	Title

($\#tables$), i.e., cross-polytopes and hyperplanes, respectively, and (ii) the number of hash functions ($\#hashes$). We tested values within the ranges reported in Table V, because further ones increased the query time to a considerable extend for a marginal increase in precision. The number of probes for multi-probe was automatically set to achieve the target recall using the approach in [63]. A parameter applying only to CrossPolytope LSH is the last cp dimension, which is chosen between 1 and the smallest power of two larger than the dimension of the embeddings vector (here 512) [64].

Among the cardinality-based dense NN methods, there are two more common parameters, which are the same as in kNN-Join (Table IV): RVS and the cardinality-threshold, K. For the latter, we consider all values in $[1, 100]$ with a step of 1, as in kNN-Join. Given, though, that this is not sufficient in some datasets, we additionally consider all values in $[105, 1000]$ with a step of 5 and all values in $[1010, 5000]$ with a step of 10. In each case, the grid search starts from the lowest value and terminates as soon as PC reaches the target recall.

FAISS does not use any other parameter apart from the common ones. Our experiments also demonstrated that it should use the Flat index, while the embedding vectors should always be normalized and combined with the Euclidean distance.

SCANN adds to the common parameters the type of index – asymmetric-hashing (AH) or brute-force (BF) – and the similarity measure – dot product or Euclidean distance. There is no clear winner among these options (cf. Table XI in [41]).

Finally, DeepBlocker adds to the common parameters the tuple embedding model. We experimented with both top-performing modules, namely AutoEncoder and Hybrid. In most cases, though, the latter raised out-of-memory exceptions, while being a whole order of magnitude slower than the former, as documented in [9]. For this reason, we exclusively consider AutoEncoder in the following.

Take-away message. All filtering techniques involve three or more configuration parameters that require fine-tuning, a non-trivial task, given that it typically involves several thousands of different settings. Some parameters are common among the techniques of the same category and, thus, experience with one approach can be useful in fine-tuning another one of the same type. Other parameters are intuitive, i.e., easily configured, such as the number of candidates per entity, which is the main parameter of cardinality-based NN methods. For this reason, these methods offer the highest usability, especially when involving a deterministic functionality. These are kNN-Join, FAISS and SCANN. Among them, only kNN-Join operates on syntactic representations, which allows for taking interpretable decisions, just like the blocking workflows. DeepBlocker uses a cardinality threshold, too, but its tuple embedding model employs neural networks with random initialization that are trained on automatically generated random synthetic data; this renders it a stochastic, and thus less robust approach. Among the similarity-based methods, only ε-Join is deterministic, while the three LSH methods are stochastic by definition: MinHash LSH involves random permutations of the input token sets, whereas Hyperplane and CrossPolytope LSH constitute random spatial partitions of a d-dimensional unit sphere. Hence, despite their smaller configuration space, ε-Joins offer higher usability.

TABLE VII
TECHNICAL CHARACTERISTICS OF THE SYNTHETIC, DIRTY ER DATASETS.

	$\mathbf{D_{10K}}$	$\mathbf{D_{50K}}$	$\mathbf{D_{100K}}$	$\mathbf{D_{200K}}$	$\mathbf{D_{300K}}$	$\mathbf{D_{1M}}$	$\mathbf{D_{2M}}$
$\|E\|$	10,000	50,000	100,000	200,000	300,000	1,000,000	2,000,000
$\|D\|$	8,705	43,071	85,497	172,403	257,034	857,538	1,716,102
$\|\|E\|\|$	$5.00{\cdot}10^7$	$1.25{\cdot}10^9$	$5.00{\cdot}10^9$	$2.00{\cdot}10^{10}$	$4.50{\cdot}10^{10}$	$5.00{\cdot}10^{11}$	$2.00{\cdot}10^{12}$

VI. QUANTITATIVE ANALYSIS

Datasets. We use two sets of datasets. The first one involves 10 real-world datasets for Clean-Clean ER that are popular in the literature [9], [11], [22], [65]. Their technical characteristics are reported in Table VI. D_1, which was first used in OAEI 2010 [66], contains restaurant descriptions. D_2 encompasses duplicate products from the online retailers Abt.com and Buy.com [65]. D_3 matches product descriptions from Amazon.com and the Google Base data API (GB) [65]. D_4 entails bibliographic data from DBLP and ACM [65]. D_5, D_6 and D_7 involve descriptions of television shows from TheTVDB.com (TVDB) and of movies from IMDb and themoviedb.org (TMDb) [67]. D_8 matches product descriptions from Walmart and Amazon [22]. D_9 involves bibliographic data from publications in DBLP and Google Scholar (GS) [65]. D_{10} interlinks movie descriptions from IMDb and DBpedia [19] – note that it includes a different snapshot of IMDb than D_5 and D_6.

The second set of datasets involves seven synthetic ones for Dirty ER of increasing size, from 10 thousand to 2 million entities. They have been widely used in the literature [11], [16], [68], as they are ideal for investigating the scalability of filtering techniques. They have been generated by Febrl [69] using the guidelines specified in [12]: first, duplicate-free entities describing persons (i.e., their names, addresses etc) were created based on frequency tables of real-world data. Then, duplicates of these entities were randomly generated according to real-world error characteristics and modifications. The resulting datasets contain 40% duplicate entities with up to 9 duplicates per entity, no more than 3 modifications per attribute, and up to 10 modifications per entity. Table VII reports their technical characteristics – $|D|$ and $||E||$ stand for the number of duplicates and the Cartesian product, resp.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

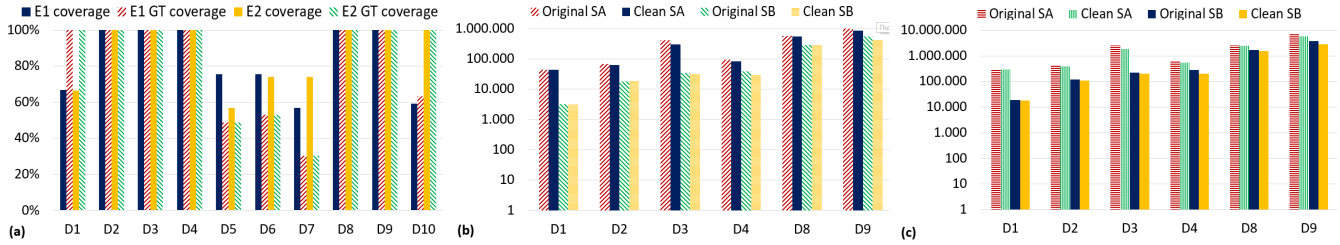


Fig. 3. (a) The coverage of the best attribute per each dataset, (b) the vocabulary size in schema-agnostic and schema-based settings, and (c) the overall character length in the textual content of the datasets for both schema settings.

Setup. All experiments on the Clean-Clean ER datasets were performed on commodity hardware equipped with an Intel i7-4710MQ @ 2.50GHz with 16GB of RAM, running Ubuntu 18.04.3 LTS. The available memory should suffice, given that all datasets occupy few MBs on the disk in their original form. All experiments on the Dirty ER datasets were performed on a server with an Intel Xeon Gold 6238R @ 2.20GHz with 128GB of RAM, running Ubuntu 18.04.6 LTS. For most time measurements, we performed 10 repetitions and report the average value. These measurements do not include the time required to load the input data into main memory.

For the implementation of all methods, we used existing, popular implementations. For blocking workflows and sparse NN methods, we employed JedAI's latest version, 3.2.1 [70]. All experiments were run on Java 15. For MH-LSH, we used `java-LSH`, version 0.12 [71]. For HP- and CP-LSH, we used the Python wrapper of FALCONN [64], version 1.3.1. For FAISS, we used version 1.7.2 of the Python wrapper provided by Facebook Research [72]. For SCANN, we used version 1.2.5 of the Python implementation provided by Google Research [73]. For DeepBlocker, we used the implementation provided by the authors [74]. FAISS, SCANN and DeepBlocker can exploit GPU optimizations, but all methods were run on a single CPU to ensure a fair comparison.

Schema settings. In each dataset, we consider both *schema-agnostic* and *schema-based settings*. The former supports heterogeneous datasets, as it takes into account the content of all attributes, regardless of their attribute names, while the latter focuses on the values of the most suitable attribute in terms of coverage and distinctiveness. We define *coverage* of attribute a as the portion of entities that contain a non-empty value for a, while *distinctiveness* expresses the portion of different values among these entities (e.g., an attribute like year for publications or movies has very low distinctiveness in contrast to their titles). Based on these criteria, we selected the attributes in Table VI for the schema-based settings.

The actual coverage of these attributes per dataset is reported in Figure 3(a) along with their *groundtruth coverage*, i.e., the portion of duplicate profiles that have at least one non-empty value for the respective attribute. We observe that for half the datasets (D_2-D_4, D_8, D_9), the selected attribute has perfect (groundtruth) coverage. In D_5-D_7, though, the overall coverage fluctuates between 55% and 75%, dropping to 30%-53% for duplicates, even though we have selected the most frequent attribute in each case. For these datasets, no filtering technique can satisfy the target recall specified in Section III. As a result, we exclude the schema-based settings of D_5-D_7 from our analysis. The same applies to D_{10}, even though the inadequate coverage pertains only to one of its constituent datasets. An exception is D_1, where the selected attribute covers just 2/3 of all profiles, but all of the duplicate ones.

Note that the low coverage for distinctive attributes like "Name" and "Title" does not mean that there are entities missing the corresponding values. Their values are typically misplaced, associated with a different attribute, e.g., due to extraction errors [9], [22]. *The schema-agnostic settings inherently tackle this form of noise, unlike the schema-based ones.*

For the datasets that have both settings, it is worth comparing their computational cost in terms of *vocabulary size* (i.e., total number of distinct tokens) and *overall character length* (i.e., total number of characters). These measures appear in Figures 3(b) and (c), resp. In each case, we also consider the values of these measures after *cleaning*, i.e., after removing the stop-words and stemming all tokens, as required by the workflow in Figure 2. We used `nltk` [75] for this purpose.

We observe that on average, the schema-based settings reduce the vocabulary size and the character length by 66.0% and 67.7%, respectively. The reason is that in most cases, the schema-agnostic settings include 3-4 name-value pairs, on average, as indicated in Table VI (av. profile). The more attributes and the more name value (n-v) pairs a dataset includes, the larger is the difference between the two settings. Cleaning further reduces the vocabulary size by 11.9% and the character length by 13.5%, on average. Hence, *the schema-based settings are expected to significantly reduce the run-time of filtering methods, especially when combined with cleaning*.

Baseline methods. To highlight the impact of fine-tuning, our analysis includes two baseline methods per type that require no parameter configuration. Instead, they employ default parameters that are common across all datasets.

In fact, we consider two baseline blocking workflows:

(i) *Parameter-free BW (PBW)*. It combines three methods with no configuration parameter (see Section IV-B): Standard Blocking, Block Purging and Comparison Propagation. It constitutes a Standard Blocking workflow with no configuration.

(ii) *Default BW (DBW)*. We experimented with the default configurations specified in [11] for the five blocking workflows discussed in Section IV-B and opted for the one achieving the best performance, on average, across all settings. This configuration is Q-Grams Blocking with $q = 6$ for block building, Block Filtering with ratio=0.5 for block cleaning and $WEP + ECBS$ for comparison cleaning.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

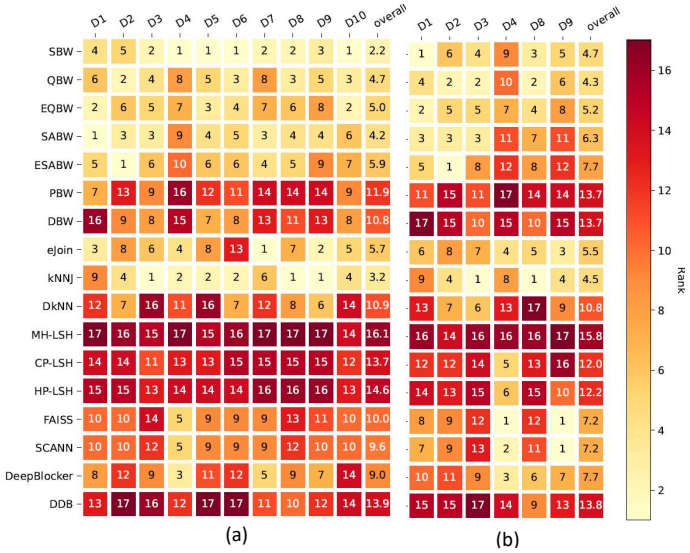


Fig. 4. The ranking position of each method per dataset in (a) schema-agnostic and (b) schema-based settings. Lower/brighter is better.

For sparse NN methods, we use the *Default kNN-Join (DkNN)* as a baseline. The reason is that kNN-Join typically outperforms the other algorithms of this type and is easy to configure, since it constitutes a deterministic, cardinality-based approach. Table X in [41] shows that its best performance is usually achieved when it is combined with cosine similarity, pre-processing to clean the attribute values and a very low number of nearest neighbors per query entity, K. We used the smallest input dataset as the query set, minimizing the candidate pairs, set K=5 and $C5GM$ as the default representation model, since it achieves the best average performance.

Among the dense NN methods, we selected DeepBlocker as the baseline approach, given that it typically outperforms all others in terms of effectiveness (cf. Figure 4). Table XI in [41] shows that it usually works best when cleaning the attribute values with stemming and stop-word removal, when using the smallest input dataset as the query set and when using a small number of candidates per query. We set K=5 so that the *Default DeepBlocker (DDB)* as with DkNN.

Schema-agnostic Settings. Due to lack of space, the detailed experimental results with respect to PC, PQ, run-time and the number of candidates over the datasets in Table VI are reported in Table VIII in [41]. All fine-tuned methods consistently exceed the target recall, i.e., $PC \geq 0.9$, regardless of their type – only DkNN and DDB violate the desired recall level in a few cases. For this reason, the relative effectiveness of the considered methods is primarily determined by PQ.

Figure 4(a) presents the ranking of every filtering technique with respect to precision (PQ) per dataset, along with its average ranking position across all datasets. The method achieving the highest PQ is ranked first, the next best second etc. Ties receive the same ranking, which is the highest possible; e.g, if FAISS and SCANN have the highest PQ after the third best method, they are both placed fourth and the next best method is placed in the sixth place. Methods that fail to satisfy the recall threshold are placed at the last ranking position.

Among the blocking workflows, the Standard Blocking one (SBW) has the highest average ranking position (2.2), because it outperforms all others in the eight largest datasets. Its parameter-free counterpart, PBW, exhibits the lowest average ranking (11.9), thus highlighting the benefits of fine-tuning. The same conclusion is drawn from the comparison between the Q-Grams Blocking workflow (QBW) and its default configuration, DBW, whose average ranking positions are 4.7 and 10.8, respectively. The Extended Q-Grams Blocking workflow (EQBW) follows QBW in close distance, taking the 5th position, on average. These three methods are outperformed by Suffix Arrays Blocking workflow (SABW), which has the second highest average ranking (4.2), while being consistently faster. Finally, the Extended Suffix Arrays Blocking workflow (ESABW) ranks as the last fine-tuned workflow, even though it achieves the overall best PQ for D_{a2}.

These patterns suggest that *attribute value tokens offer the best granularity for blocking signatures in the schema-agnostic settings*. Even though some candidates might be missed by typographical errors, they typically share multiple other tokens, due to the schema-agnostic settings. Using substrings of tokens (i.e., q-grams and suffix arrays) as signatures increases significantly the number of candidate pairs, without any benefit in recall. These pairs are significantly reduced by the block and comparison cleaning, but to a lesser extent than those of SBW, yielding lower precision. The only exceptions are the two smallest datasets, where the maximum block size limit of (E)SABW raises precision to the overall highest level.

Among the sparse NN methods, we observe that kNN-Join (kNNJ) outperforms ϵ-Join in eight datasets. In half of these cases, kNNJ actually achieves the best precision among all considered methods. On average, kNNJ ranks much higher than ϵ-Join (3.2 vs 5.7, respectively), which suggests that *the cardinality thresholds are significantly more effective in reducing the search space of ER than the similarity ones*. The reason is that the latter apply a global condition, unlike the former, which operate locally, selecting the best candidates per query entity. Comparing kNNJ with its baseline method, DkNN, the former consistently outperforms the latter, as expected, verifying the benefits of parameter fine-tuning.

Regarding the dense NN methods, we observe that the similarity-based ones consistently achieve the lowest by far precision among all fine-tuned techniques. CP-LSH typically outperforms MH- and HP-LSH, but underperforms all baseline methods (i.e., PBW, DBW and DkNN) in most of the cases, especially over the largest datasets. The reason is that *the similarity-based methods achieve high recall only by producing an excessively large number of candidate pairs* (MH-LSH actually runs out of memory when processing D_{a10}). Their precision raises to high levels only for $PC \ll 0.9$. This applies to both sparse syntactic and dense embedding vectors.

Significantly better performance is achieved by the cardinality-based NN methods. FAISS and SCANN exhibit practically identical performance across all datasets, because they perform an exhaustive search of the nearest neighbors. They differ only in D_{a3}, D_{a8} and D_{a9}, where SCANN

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

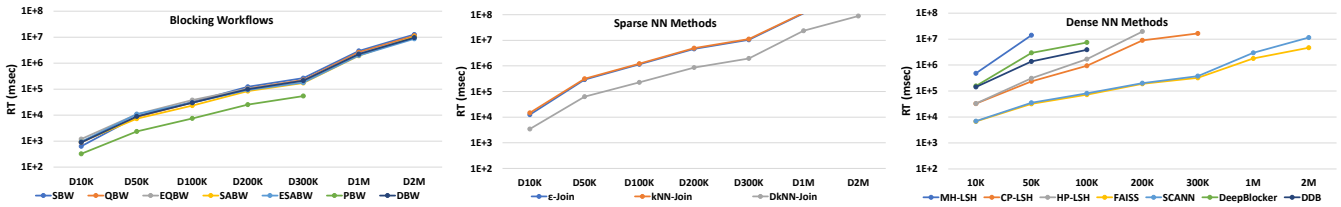


Fig. 5. Scalability analysis with respect to run-time (in milliseconds) over all datasets in Table VII. Note the logarithmic scale on the vertical axis.

outperforms FAISS, despite using approximate scoring (AH). The two algorithms outperform all NN methods in four datasets, with DeepBlocker being the top performer in the remaining six. As a result, DeepBlocker exhibits the highest average ranking position among all methods of this type (9.0), which means that *the learning-based tuple embedding module raises significantly the precision of NN methods*. However, DeepBlocker does not scale to D_{a10} with the available memory resources, due to the extremely large set of candidate pairs. The same applies to its default configuration, DDB. Note that DDB fails to achieve the target recall in four datasets; for the remaining five, its low average ranking position verifies the need for parameter fine-tuning.

Comparing the top performing methods from each category in terms of precision, we notice that SBW takes a clear lead (2.2), followed in close distance by kNNJ (3.2), leaving DeepBlocker in the last place (9.0). SBW achieves the maximum PQ in four datasets, kNNJ in three and DeepBlocker in none of them. Note, though, that *kNNJ constitutes a more robust approach that is easier to configure and apply in practice*. Its default configuration, DkNN, exhibits the highest average ranking position, together with DBW (10.9 and 10.8, respectively), outperforming the other two baseline methods to large extent: PBW ranks 11.9 and DDB 13.9.

Schema-based settings. Similar to the schema-agnostic settings, all fine-tuned filtering methods achieve the target PC. The baseline methods fail in two datasets, except for DkNN, which fails just once. Along with the four datasets with insufficient coverage, this means that *without fine-tuning, the schema-based settings fall short of recall in half the cases*.

Regarding precision, SBW and QBW outperform all blocking workflows in two datasets each, but the latter achieves the highest average ranking position (4.3), leaving SBW in the second place with 4.7. EQBW ranks third (5.2) and SABW fourth (6.3), even though each method is the top performer in one dataset. ESABW again exhibits the lowest average ranking position among all fine-tuned workflows.

Among the sparse NN methods, there is a balance between ϵ-Join and kNNJ, as each method achieves the top precision in half the datasets. Yet, kNNJ lies very close to ϵ-Join in the cases where the latter is the top performer, but not vice versa: as a result, its average ranking position (4.5) is much higher than that of ϵ-Join (5.5). DkNN exhibits a robust, high performance that remains very close to kNNJ in all datasets, except D_{b8}, where it fails to reach the target recall. Note that DkNN outperforms ϵ-Join over D_{b2} and D_{b3}. These settings verify that *the cardinality thresholds are superior to the similarity ones, regardless of the schema settings*.

Regarding the dense NN methods, the similarity-based ones, i.e., the LSH variants, consistently underperform the cardinality-based ones. MH-LSH actually does not scale to D_{b9}, due to very large set of candidates it produces. FAISS and SCANN exhibit practically identical performance, outperforming DeepBlocker in four datasets. As a result, they achieve a slightly higher average ranking position (7.2 vs 7.7). The baseline method DDB exhibits low precision, merely outperforming MH-LSH.

Among the top performing fine-tuned methods per category, *QBW and kNNJ exhibit the best and most robust performance*. The latter actually achieves the overall best PQ in two datasets. Both methods outperform FAISS and SCANN to a significant extent, judging from their average ranking positions (4.3 and 4.5 vs 7.2). Note that DkNN outperforms all other baselines in most cases, achieving the highest by far average ranking position (10.8 vs 13.7 and 13.8).

Scalability Analysis. We now examine the relative time efficiency of all filtering techniques as the size of the input data increases, using the seven synthetic datasets in Table VII. Note that the different programming languages do not allow for comparing them on an equal basis. For this reason, we follow the approach of ANN Benchmark [15], which compares implementations rather than algorithms. The reason is that even slight changes (e.g., a different data structure) in the implementation of the same algorithm in the same language might lead to significantly different run-times.

Figure 5 reports the experimental results. Note that the scale of the vertical axis is logarithmic, with the maximum value (10^8 msec) corresponding to 27.8 hrs. Every filtering technique was fine-tuned on the smallest dataset (D_{10K}) with respect to Problem 1 and the same configuration was applied to all seven datasets. We exclusively considered schema-agnostic settings, due to their robustness with respect to recall. The exact configurations are reported in Tables IX-XI in [41].

Starting with the blocking workflows on the left, we observe that PBW is the fastest one, due to its simple comparison cleaning, which merely applies Comparison Propagation to eliminate the redundant candidate pairs. As a result, it does not scale to the two largest datasets, D_{1M} and D_{2M}, due to the very large number of candidate pairs it generates. All other workflows are coupled with a Meta-blocking approach that assigns a weight to every candidate pair and prunes the lowest-weighted ones in an effort to reduce the superfluous pairs, too. As a result, they trade higher run-times for higher scalability. The differences between most blocking workflows are minor: ESABW is the fastest and SBW the slowest one, requiring 2.4 and 3.5 hrs, respectively, over D_{2M}.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

Among the sparse NN methods, only DkNN scales to D_{2M}, because its large q-grams ($q = 5$) generate few candidates per entity. In contrast, ϵ-Join and kNNJ rely on character bigrams, yielding a time-consuming functionality, due to the excessive number of candidates: they require more than 30 hrs for D_{1M}.

Among the dense NN methods, MH-, HP- and CP-LSH scale up to D_{50K}, D_{200K} and D_{300K}, respectively, because their large number of candidates does not fit into the available main memory. Similarly, DeepBlocker and DDB scale up to D_{100K}, because of their quadratic time complexity (i.e., they requires ∼300GB of RAM to create a 200K×200K float array for D_{200K}). FAISS and SCANN exhibit much higher scalability, due to the approximate indexes they employ (IVF and AH, respectively). The former actually processes D_{2M} within 1.3 hrs, being the fastest approach by far, while the latter exhibits run-times similar to the blocking workflows, requiring 3.2 hrs for D_{2M}.

VII. Conclusions

Our experimental results lead to the following conclusions:

1) Fine-tuning vs default parameters. For all types of methods, optimizing the internal parameters with respect to a performance goal significantly raises the performance of filtering. This problem is poorly addressed in the literature [10], and the few proposed tuning methods require the involvement of experts [76], [77]. *More emphasis should be placed on a-priori fine-tuning the filtering methods through an automatic, data-driven approach that requires no labelled set*.

2) Schema-based vs schema-agnostic settings. The former significantly improve the time efficiency at the cost of unstable effectiveness, while the latter offer robust effectiveness, as they inherently address heterogeneous schemata as well as misplaced and missing values that are common in ER [9], [22]. Even when the schema-based settings exhibit high recall, their maximum precision outperforms the schema-agnostic settings in just three cases (D_1–D_3). The schema-agnostic settings also exceed the target recall even in combination with default configurations (the baseline blocking workflows), unlike the schema-based settings, where all baseline methods fall short of the target recall at least once. For these reasons, *the schema-agnostic settings are preferable over the schema-based ones*.

3) Similarity vs cardinality thresholds. Poor performance is typically achieved by all similarity-based NN methods (cf. Table II). The LSH variants achieve high recall only by producing an excessively large number of candidates: MH-, CP- and HP-LSH reduce the candidate pairs of the brute-force approach by 48%, 91% and 89%, respectively, on average, across all datasets in Table VI. This might seem high (a whole order of magnitude for CP- and HP-LSH), but is consistently inferior to the cardinality-based NN methods. This applies even to ε-Join, which is the best similarity-based approach, reducing the candidate pairs of the brute-force approach by 99% (i.e., multiple orders of magnitude): it underperforms kNN-Join in 9 out of 16 cases. Most importantly, the number of candidates produced by similarity-based methods depends quadratically on the total size of the input. For the cardinality-based methods, it depends linearly on the size of the query dataset, which is usually the smallest one, i.e., $|C| = k \cdot min(|E_1|, |E_2|)$; typically, $k \ll 100$ for cardinality-based methods, especially for kNN-Join. Therefore, *cardinality thresholds are preferable over similarity thresholds.*

4) Syntactic vs semantic representations. The blocking workflows and the sparse NN methods assume that the pairs of duplicates share textual content; the rarer this content is, the more likely are two entities to be matching. In contrast, most dense NN methods assume that the duplicate entities share syntactically different, but semantically similar content that can be captured by pre-trained character-level embeddings. The latter assumption is true for the matching step of ER [9], but our experiments advocate that filtering violates this assumption: semantic-based representations outperform the syntactic ones only in two cases (D_{b4} and D_{b9}). Comparing kNN-Join with cardinality-based dense NN methods, we observe that the former consistently uses a lower threshold (see Tables X and XI in [41]). This means that the semantic representations introduce more false positives than the syntactic ones, due to the out-of-vocabulary, domain-specific terms in ER datasets – see [41] for an in depth analysis. Hence, the *syntactic representations are preferable over semantic ones*.

5) Most effective filtering method. The only method that combines a cardinality threshold with a syntactic representation is kNN-Join. Although the Standard Blocking workflow (SBW) performs better in the schema-agnostic settings, kNN-Join offers two qualitative advantages: (i) Unlike SBW, the number of candidates is linear in the input size. (ii) kNN-Join is easy to configure as shown by the high performance of the DkNN-Join baseline: even though its recall fluctuates in $[0.8, 0.9]$ for three datasets, it outperforms PBW, the default configuration of SBW, in almost all other cases.

6) Most scalable filtering method. Only the blocking workflows, FAISS and SCANN, scale to all synthetic, Dirty ER datasets within the 4 hours timeout. As the number of input entities increases from 10^4 (D_{10K}) to $2{\cdot}10^6$ (D_{2M}), the run-times of all techniques scale superlinearly (>200 times), but subquadratically (<40,000 times). For the blocking workflows, the increase ranges from ∼8,000 (EQBW) to ∼20,000 times (SBW). The factor is just ∼1,600 for SCANN and ∼700 for FAISS. Thus, FAISS is by far the fastest and most scalable filtering technique for large datasets, due to its approximate indexing scheme, leaving SCANN in the second place.

In the future, we will enrich the *Continuous Benchmark of Filtering methods for ER* with new datasets and will update the rankings per dataset with new filtering methods.

Acknowledgements. This research was partially funded by the Austrian Science Fund (FWF) P 34962, EU Horizon Europe GA No 101070122 (STELAR), the Hellenic Foundation for Research and Innovation (Project Number: HFRI-FM17-2351 GeoQA) and the European Commission through the xAIM project, agreement No INEA/CEF/ICT/A2020/ 2276680. For the purpose of open access, the authors have applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] L. Getoor and A. Machanavajjhala, "Entity resolution: Theory, practice & open challenges," *Proc. VLDB Endow.*, vol. 5, no. 12, pp. 2018–2019, 2012.

[2] X. L. Dong and D. Srivastava, *Big Data Integration*, ser. Synthesis Lectures on Data Management. Morgan & Claypool Publishers, 2015.

[3] P. Christen, *Data Matching - Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*, ser. Data-Centric Systems and Applications. Springer, 2012.

[4] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios, "Duplicate record detection: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 19, no. 1, pp. 1–16, 2007.

[5] V. Christophides, V. Efthymiou, and K. Stefanidis, *Entity Resolution in the Web of Data*, ser. Synthesis Lectures on the Semantic Web: Theory and Technology. Morgan & Claypool Publishers, 2015.

[6] G. Papadakis, E. Ioannou, E. Thanos, and T. Palpanas, *The Four Generations of Entity Resolution*, ser. Synthesis Lectures on Data Management. Morgan & Claypool Publishers, 2021.

[7] N. Barlaug and J. A. Gulla, "Neural networks for entity matching: A survey," *ACM Trans. Knowl. Discov. Data*, vol. 15, no. 3, pp. 52:1–52:37, 2021.

[8] O. Hassanzadeh, F. Chiang, R. J. Miller, and H. C. Lee, "Framework for evaluating clustering algorithms in duplicate detection," *Proc. VLDB Endow.*, vol. 2, no. 1, pp. 1282–1293, 2009.

[9] S. Thirumuruganathan, H. Li, N. Tang, M. Ouzzani, Y. Govind, D. Paulsen, G. Fung, and A. Doan, "Deep learning for blocking in entity matching: A design space exploration," *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2459–2472, 2021.

[10] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Comput. Surv.*, vol. 53, no. 2, pp. 31:1–31:42, 2020.

[11] G. Papadakis, J. Svirsky, A. Gal, and T. Palpanas, "Comparative analysis of approximate blocking techniques for entity resolution," *Proc. VLDB Endow.*, vol. 9, no. 9, pp. 684–695, 2016.

[12] P. Christen, "A survey of indexing techniques for scalable record linkage and deduplication," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 9, pp. 1537–1555, 2012.

[13] W. Mann, N. Augsten, and P. Bouros, "An empirical evaluation of set similarity join techniques," *Proc. VLDB Endow.*, vol. 9, no. 9, pp. 636–647, 2016.

[14] Y. Jiang, G. Li, J. Feng, and W. Li, "String similarity joins: An experimental evaluation," *Proc. VLDB Endow.*, vol. 7, no. 8, pp. 625–636, 2014.

[15] M. Aumüller, E. Bernhardsson, and A. J. Faithfull, "Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Inf. Syst.*, vol. 87, 2020.

[16] G. Papadakis, G. Alexiou, G. Papastefanatos, and G. Koutrika, "Schema-agnostic vs schema-based configurations for blocking methods on homogeneous data," *Proc. VLDB Endow.*, vol. 9, no. 4, pp. 312–323, 2015.

[17] F. Fier, N. Augsten, P. Bouros, U. Leser, and J. Freytag, "Set similarity joins on mapreduce: An experimental survey," *Proc. VLDB Endow.*, vol. 11, no. 10, pp. 1110–1122, 2018.

[18] G. Papadakis, L. Tsekouras, E. Thanos, G. Giannakopoulos, T. Palpanas, and M. Koubarakis, "Domain- and structure-agnostic end-to-end entity resolution with jedai," *SIGMOD Rec.*, vol. 48, no. 4, pp. 30–36, 2019.

[19] G. Papadakis, G. M. Mandilaras, L. Gagliardelli, G. Simonini, E. Thanos, G. Giannakopoulos, S. Bergamaschi, T. Palpanas, and M. Koubarakis, "Three-dimensional entity resolution with jedai," *Inf. Syst.*, vol. 93, p. 101565, 2020.

[20] M. G. Elfeky, A. K. Elmagarmid, and V. S. Verykios, "TAILOR: A record linkage tool box," in *ICDE*, 2002, pp. 17–28.

[21] P. Konda, S. Das, P. S. G. C., A. Doan, A. Ardalan, J. R. Ballard, H. Li, F. Panahi, H. Zhang, J. F. Naughton, S. Prasad, G. Krishnan, R. Deep, and V. Raghavendra, "Magellan: Toward building entity matching management systems," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 1197–1208, 2016.

[22] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra, "Deep learning for entity matching: A design space exploration," in *SIGMOD*. ACM, 2018, pp. 19–34.

[23] S. Galhotra, D. Firmani, B. Saha, and D. Srivastava, "Efficient and effective ER with progressive blocking," *VLDB J.*, vol. 30, no. 4, pp. 537–557, 2021.

[24] ——, "BEER: blocking for effective entity resolution," in *SIGMOD*, 2021, pp. 2711–2715.

[25] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to information retrieval*. Cambridge University Press, 2008.

[26] S. Wang, W. Zhou, and C. Jiang, "A survey of word embeddings based on deep learning," *Computing*, vol. 102, no. 3, pp. 717–740, 2020.

[27] G. Papadakis, E. Ioannou, T. Palpanas, C. Niederée, and W. Nejdl, "A blocking framework for entity resolution in highly heterogeneous information spaces," *IEEE Trans. Knowl. Data Eng.*, vol. 25, no. 12, pp. 2665–2682, 2013.

[28] G. Papadakis, G. Koutrika, T. Palpanas, and W. Nejdl, "Meta-blocking: Taking entity resolution to the next level," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 8, pp. 1946–1960, 2014.

[29] G. Simonini, S. Bergamaschi, and H. V. Jagadish, "BLAST: a loosely schema-aware meta-blocking approach for entity resolution," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 1173–1184, 2016.

[30] L. Gravano, P. G. Ipeirotis, H. V. Jagadish, N. Koudas, S. Muthukrishnan, and D. Srivastava, "Approximate string joins in a database (almost) for free," in *VLDB*, 2001, pp. 491–500.

[31] N. Augsten and M. H. Böhlen, *Similarity Joins in Relational Database Systems*. Morgan & Claypool Publishers, 2013.

[32] N. Augsten, "A roadmap towards declarative similarity queries," in *EDBT*, 2018, pp. 509–512.

[33] Y. N. Silva, W. G. Aref, P. Larson, S. Pearson, and M. H. Ali, "Similarity queries: their conceptual evaluation, transformations, and processing," *VLDB J.*, vol. 22, no. 3, pp. 395–420, 2013.

[34] R. J. Bayardo, Y. Ma, and R. Srikant, "Scaling up all pairs similarity search," in *WWW*, 2007, pp. 131–140.

[35] S. Chaudhuri, V. Ganti, and R. Kaushik, "A primitive operator for similarity joins in data cleaning," in *ICDE*, 2006, p. 5.

[36] P. Bouros, S. Ge, and N. Mamoulis, "Spatio-textual similarity joins," *Proc. VLDB Endow.*, vol. 6, no. 1, pp. 1–12, 2012.

[37] D. Deng, G. Li, H. Wen, and J. Feng, "An efficient partition based method for exact set similarity joins," *Proc. VLDB Endow.*, vol. 9, no. 4, pp. 360–371, 2015.

[38] D. Deng, Y. Tao, and G. Li, "Overlap set similarity joins with theoretical guarantees," in *SIGMOD*, 2018, pp. 905–920.

[39] E. Zhu, D. Deng, F. Nargesian, and R. J. Miller, "JOSIE: overlap set similarity search for finding joinable tables in data lakes," in *SIGMOD*, 2019, pp. 847–864.

[40] C. Xiao, W. Wang, X. Lin, J. X. Yu, and G. Wang, "Efficient similarity joins for near-duplicate detection," *ACM Trans. Database Syst.*, vol. 36, no. 3, pp. 15:1–15:41, 2011.

[41] G. Papadakis, M. Fisichella, F. Schoger, G. Mandilaras, N. Augsten, and W. Nejdl, "How to reduce the search space of entity resolution: with blocking or nearest neighbor search?" https://arxiv.org/abs/2202.12521, 2022.

[42] C. Li, J. Lu, and Y. Lu, "Efficient merging and filtering algorithms for approximate string searches," in *ICDE*, 2008, pp. 257–266.

[43] D. Kocher and N. Augsten, "A scalable index for top-k subtree similarity queries," in *SIGMOD*, 2019, pp. 1624–1641.

[44] C. Xiao, W. Wang, X. Lin, and H. Shang, "Top-k set similarity joins," in *ICDE*, 2009, pp. 916–927.

[45] Z. Yang, B. Zheng, G. Li, X. Zhao, X. Zhou, and C. S. Jensen, "Adaptive top-k overlap set similarity joins," in *ICDE*, 2020, pp. 1081–1092.

[46] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *STOC*, 1998, p. 604–613.

[47] M. Fisichella, A. Ceroni, F. Deng, and W. Nejdl, "Predicting pair similarities for near-duplicate detection in high dimensional spaces," in *DEXA*, 2014, pp. 59–73.

[48] D. Karapiperis, D. Vatsalan, V. S. Verykios, and P. Christen, "Efficient record linkage using a compact hamming space," in *EDBT*, 2016, pp. 209–220.

[49] H. Kim and D. Lee, "HARRA: fast iterative hashed record linkage for large-scale data collections," in *EDBT*, 2010, pp. 525–536.

[50] W. Zhang, H. Wei, B. Sisman, X. L. Dong, C. Faloutsos, and D. Page, "Autoblock: A hands-off blocking framework for entity matching," in *WSDM*, 2020, pp. 744–752.

[51] M. Ebraheem, S. Thirumuruganathan, S. R. Joty, M. Ouzzani, and N. Tang, "Distributed representations of tuples for entity resolution," *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1454–1467, 2018.

[52] A. Z. Broder, "On the resemblance and containment of documents," in *SEQUENCES*, 1997, pp. 21–29.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.

[53] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of massive data sets*. Cambridge university press, 2020.
[54] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *STOC*, 2002, pp. 380–388.
[55] B. N. et al., "Multiprobe-lsh," https://github.com/gopalmenon/Multi-Probe-LSH, 2018.
[56] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
[57] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *ICML*, 2020, pp. 3887–3896.
[58] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 135–146, 2017.
[59] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra, "Deep learning for entity matching: A design space exploration," in *SIGMOD*, 2018, pp. 19–34.
[60] C. Fu, X. Han, J. He, and L. Sun, "Hierarchical matching network for heterogeneous entity resolution," in *IJCAI*, 2020, pp. 3665–3671.
[61] Z. Yao, C. Li, T. Dong, X. Lv, J. Yu, L. Hou, J. Li, Y. Zhang, and Z. Dai, "Interpretable and low-resource entity matching via decoupling feature learning from decision making," in *ACL/IJCNLP*, 2021, pp. 2770–2781.
[62] D. Zhang, Y. Nie, S. Wu, Y. Shen, and K. Tan, "Multi-context attention for entity matching," in *WWW*, 2020, pp. 2634–2640.
[63] FALCONN-LIB, "Fine-tuning Multi-proble LSH," https://github.com/FALCONN-LIB/FALCONN/blob/master/src/examples/glove/glove.py, 2017, [Online; accessed April 25, 2022].
[64] A. Andoni, P. Indyk, T. Laarhoven, I. P. Razenshteyn, and L. Schmidt, "Practical and optimal LSH for angular distance," in *NIPS*, 2015, pp. 1225–1233, available at https://github.com/FALCONN-LIB/FALCONN.
[65] H. Köpcke, A. Thor, and E. Rahm, "Evaluation of entity resolution approaches on real-world match problems," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 484–493, 2010.
[66] J. Euzenat, A. Ferrara, C. Meilicke, J. Pane, F. Scharffe, P. Shvaiko, H. Stuckenschmidt, O. Sváb-Zamazal, V. Svátek, and C. T. dos Santos, "Results of the ontology alignment evaluation initiative 2010," in *Proceedings of the 5th International Workshop on Ontology Matching (OM-2010), Shanghai, China, November 7, 2010*, vol. 689, 2010.
[67] D. Obraczka, J. Schuchart, and E. Rahm, "Embedding-assisted entity resolution for knowledge graphs," in *ESWC*, vol. 2873, 2021.
[68] B. Kenig and A. Gal, "Mfiblocks: An effective blocking algorithm for entity resolution," *Inf. Syst.*, vol. 38, no. 6, pp. 908–926, 2013.
[69] P. Christen, "Febrl -: an open source data cleaning, deduplication and record linkage system with a graphical user interface," in *KDD*, 2008, pp. 1065–1068.
[70] JedAI, "Java gEneric DAta Integration (JedAI) Toolkit," https://github.com/scify/JedAIToolkit, 2017, [Online; accessed April 25, 2022].
[71] T. D. et al., "A java implementation of locality sensitive hashing," https://github.com/tdebatty/java-LSH, 2015-2019.
[72] F. Research, "Faiss: a library for efficient similarity search," https://github.com/facebookresearch/faiss.
[73] G. Research, "Scann," https://github.com/google-research/google-research/tree/master/scann.
[74] QCRI, "Deepblocker," https://github.com/qcri/DeepBlocker.
[75] S. Bird, E. Klein, and E. Loper, *Natural language processing with Python: analyzing text with the natural language toolkit*. O'Reilly Media, Inc., 2009.
[76] H. Li, P. Konda, P. S. G. C., A. Doan, B. Snyder, Y. Park, G. Krishnan, R. Deep, and V. Raghavendra, "Matchcatcher: A debugger for blocking in entity matching," in *EDBT*, 2018, pp. 193–204.
[77] R. Maskat, N. W. Paton, and S. M. Embury, "Pay-as-you-go configuration of entity resolution," *Trans. Large Scale Data Knowl. Centered Syst.*, vol. 29, pp. 40–65, 2016.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:57:37 UTC from IEEE Xplore. Restrictions apply.